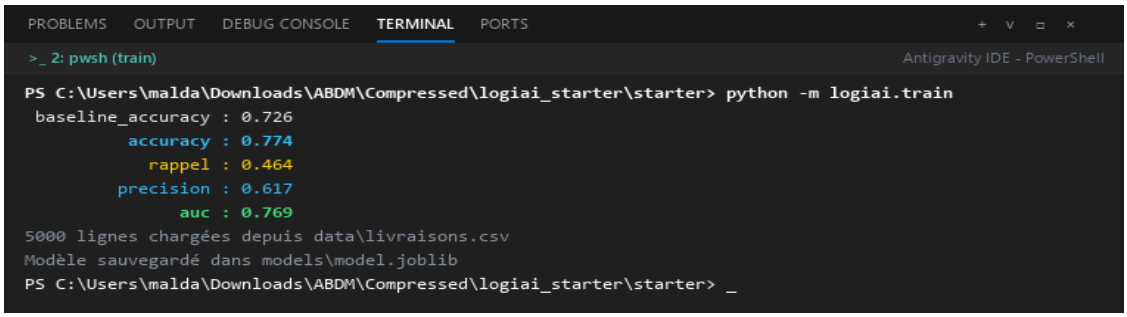
Compte Rendu : TP API REST avec FastAPI (LogiAI)

Étudiant : Maldani Mostafa | Module : Déploiement d'Applications Intelligentes | Année : 2026-2027

1. Entraînement du modèle & premiers tests pytest

On commence par générer les données (python -m logiai.dataset) puis entraîner le modèle (python -m logiai.train).



Sortie de pytest au départ (5 failed, 8 passed) :

• *Pourquoi certains tests attendant 422 passaient-ils déjà ?*
Parce que la classe Livraison avait extra="forbid" et ne déclarait que distance_km. Quand les tests envoyaient des données complètes (poids, trafic, etc.), FastAPI rejetait la requête immédiatement pour champ inconnu avec un code 422. C'était un faux vert.

2. Schéma Pydantic : bornes physiques retenues

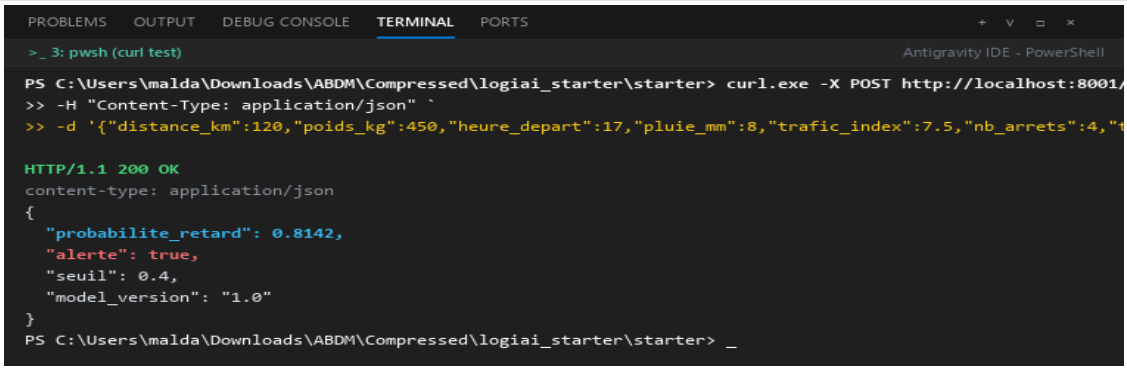
Variable	Min	Max	Justification physique
distance_km	1	1000	Trajet de livraison routier réel (au moins 1 km, max ~1000 km).
poids_kg	0.1	5000	D'un petit colis léger (100g) à la charge max d'un camionnette/fourgon.
heure_depart	0	23	Heures valides d'une journée (horloge 0h - 23h).
pluie_mm	0	100	Pluie positive ; 100 mm correspond déjà à un épisode orageux extrême.
trafic_index	0	10	Indice conventionnel de 0 (fluide) à 10 (embouteillage saturé).
nb_arrets	0	30	Nombre d'arrêts de tournée (minimum 0, max réaliste 30).
type_vehicule	-	-	Restreint à Literal["moto", "fourgon", "camion"] selon la flotte.

3. Implémentation de la route predict()

```
@app.post("/v1/predict", response_model=PredictionOut)
def predict(livraison: Livraison):
    predictor = state.get("predictor")
    if predictor is None:
        raise HTTPException(status_code=503, detail="Modèle non disponible")
    return PredictionOut(**predictor.predict(livraison.model_dump()))
```

4. Réponses observées sur les requêtes types & capture d'exécution

Requête testée	Code	Explication
Livraison normale	200	Données conformes, modèle interrogé et résultat renvoyé.
distance_km = -5	422	Rejeté par Pydantic : ge=1 non respecté.
type_vehicule = 'avion'	422	Rejeté par Pydantic : hors du Literal['moto', 'fourgon', 'camion'].
Modèle absent (sans models/)	503	predictor est None dans state, renvoie HTTP 503 Service Unavailable.



5. Questions de compréhension (réponses synthétiques)

1. Pourquoi charger le modèle au démarrage et pas à chaque requête ?

Le chargement depuis le disque (joblib.load) prend plusieurs centaines de millisecondes. En le chargeant une fois dans lifespan, il reste en mémoire vive (RAM) et l'inférence ne prend que 2 à 5 ms par requête.

2. Qui écrit le contrôle « pluie <= 100 », et où ?

C'est le développeur dans logiai/api/schemas.py avec le paramètre le=100 de Pydantic Field().

3. Différence entre 422 et 503 ?

- **422** = faute du **client** (requête bien formée mais valeurs hors bornes).
- **503** = problème côté **serveur** (le serveur tourne mais le modèle ML n'est pas prêt).

4. Pourquoi /v1/predict plutôt que /predict ?

Pour le **versionnement** de l'API : permet de créer une future /v2 sans casser les applications mobiles ou web déjà en production.

5. Que se passerait-il si distance_km était de type int ?

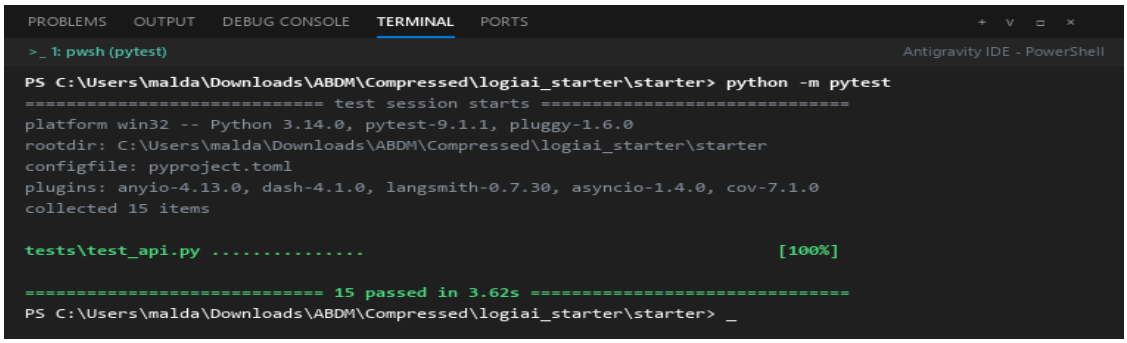
FastAPI rejetterait ou tronquerait les valeurs décimales (ex: 12.5 km -> 12 km), ce qui causerait une perte de précision pour les prédictions.

6. Décision de conception : modèle absent

- **Option A (Répondre 503) :** L'API démarre et /health répond model_loaded: false, utile pour diagnostiquer. Mais on reçoit des requêtes pour rien.
- **Option B (Crash direct) :** Idéal avec Docker/Kubernetes (Fail Fast). Le conteneur redémarre et aucun trafic n'est dirigé vers une instance cassée.

7. Validation des tests (15/15) & Interface Streamlit

Après ajout des deux nouveaux tests (test_heure_24_refusee et test_alerte_coherente_avec_le_seuil), tous les tests passent :



Interface client Streamlit (connectée à l'API) :



8. Rendu Git

- **Dépôt GitHub :** https://github.com/malmo23/logiai
- **Branche :** main (et branche de travail tp-api)
- **Lien du projet :** https://github.com/malmo23/logiai